# SIH Quantum Computing Platform

Product Research, Approach & Differentiation

Team Working Document

September 7, 2026

## Executive Summary

The SIH problem statement asks for an AI-based interactive quantum algorithm learning platform. The requested system combines learning content, a visual quantum-circuit builder, code-based circuit design, simulation through quantum-computing frameworks, state/circuit visualization, AI assistance, assessment, progress tracking, instructor analytics, and cloud deployment.

Our proposed strategy is **not** to compete directly with existing quantum IDEs or simply build "a quantum simulator with a chatbot." Instead, we should make the core product an **interactive quantum experimentation and learning environment**.

**Learn → Predict → Build → Run → Observe → Explain → Debug → Challenge → Master**

The goal is to make students actively reason about quantum circuits rather than only read explanations or execute code.

## 1 What the SIH Problem Statement Demands

The PS describes a platform intended to make quantum computing more accessible and hands-on. Its expected functionality includes:

- Learning content covering quantum fundamentals and algorithms such as Deutsch–Jozsa, Grover, QAOA and VQE.
- A visual drag-and-drop quantum circuit designer.
- A code editor for writing quantum programs.
- Simulation and integration with frameworks/backends including Qiskit Aer, PennyLane, Cirq and qBraid.
- Visualization through circuit diagrams, measurement histograms, statevectors and Bloch spheres.
- AI assistance for explanations, code generation, debugging, error detection and learning support.
- Quizzes, coding challenges and automated grading.
- Learner progress tracking and instructor-facing analytics.
- A complete responsive web application with authentication, APIs and cloud deployment.

## 2 Competitive Reality

A major strategic point is that many individual features in the PS already exist elsewhere. Therefore, simply implementing the checklist is unlikely to be a strong differentiator.

| **Platform** | **Strong at** | **Implication for us** |
|---|---|---|
| IBM Quantum Composer / Learning | Circuit construction, simulation, visualization and quantum education | A circuit builder and standard visualizations alone are not novel. |
| Microsoft Quantum Katas | Interactive lessons, coding exercises, progress, hints and AI/Copilot-assisted learning | AI + lessons + coding is already established. |
| PennyLane | Tutorials, demos, Codebook material and coding challenges | Quantum education and practice are already well covered. |
| Quirk | Very intuitive visual circuit construction and immediate simulation/state visualization | The UX should emphasize experimentation and immediate feedback. |
| Classiq | AI-assisted quantum development and circuit generation/optimization | "AI for quantum" by itself is not a sufficient USP. |

## 3 Feature Comparison Matrix

Legend: Strong = significant part of the product; Partial = present but not the central focus; Opportunity = an area we can emphasize.

| Feature | IBM | MS | PennyLane | Quirk | Classiq | Our opportunity |
|---|---|---|---|---|---|---|
| Quantum courses | Strong | Strong | Strong | No | Partial | Beginner-first experiments |
| Interactive coding | Strong | Strong | Strong | No | Strong | Context-aware practice |
| Drag/drop circuits | Strong | Partial | Partial | Strong | Strong | Part of learning loop |
| Simulation | Strong | Strong | Strong | Strong | Strong | Unified learning experience |
| State visualization | Strong | Partial | Strong | Strong | Partial | Tie visuals to concepts |
| Coding challenges | Partial | Strong | Strong | No | Partial | Circuit-aware assessment |
| AI tutor | Partial | Strong | Partial | No | Strong | Tutor understands circuit + output |
| Progress tracking | Partial | Strong | Partial | No | Partial | Concept-level mastery |
| Instructor analytics | Partial | Partial | No | No | No | Misconception/error analytics |
| Multi-framework | Partial | No | Strong | No | Strong | Common circuit representation |
| Circuit-aware AI | Partial | Partial | Partial | No | Strong | Deep circuit + simulator context |
| Learn/Build/Run/Explain | Partial | Strong | Partial | No | No | Make this the core product loop |
| Circuit debugging tutor | Partial | Partial | Partial | No | Partial | Strong WOW candidate |
| What changed? | No | No | No | Partial | No | Before/after explanation |
| Quantum experiment lab | No | Partial | Partial | Strong | No | Experiment-first pedagogy |
| Noise experimentation | Strong | Partial | Strong | Partial | Strong | Interactive Noise Lab |
| Classical vs quantum | Partial | Strong | Partial | No | No | Interactive comparison |

# 4 Proposed Product Approach

The core product should treat an **experiment**, rather than a lesson or simulator session, as the basic unit of learning.

### Example: Learning Entanglement

1. **Learn:** Explain qubits, superposition and entanglement with a small interactive example.
2. **Predict:** Ask the learner what measurement probabilities they expect before running the circuit.
3. **Build:** Let the learner construct the circuit using drag-and-drop gates or code.
4. **Run:** Execute the circuit on a simulator.
5. **Observe:** Display the circuit, measurement histogram, statevector and appropriate state visualization.
6. **Explain:** Have the AI explain the observed result using the actual circuit and simulation output.
7. **Debug:** If the result is wrong, identify likely circuit mistakes and guide the learner.
8. **Challenge:** Ask the learner to modify or create a circuit satisfying a target behavior.
9. **Master:** Record performance and update concept-level progress.

# 5 Potential Differentiating Features

### 1. Prediction Before Execution

Before running a circuit, the learner predicts the state or measurement distribution. The platform compares the prediction with the actual result and explains the discrepancy.

### 2. What Changed?

When a learner modifies a circuit, show before/after circuit behavior, state/probability changes, and a concise causal explanation.

### 3. Quantum Circuit Debugger

Give learners intentionally broken circuits or analyze their incorrect circuits. Explain the discrepancy and guide them toward a fix rather than simply giving the answer.

### 4. Noise Lab

Allow learners to vary noise/error levels and observe how ideal outcomes degrade, connecting textbook algorithms with realistic quantum limitations.

### 5. Circuit-Aware AI Tutor

The AI receives the current circuit, code, simulation output, current lesson and learner history instead of operating as a generic chatbot.

### 6. Classical vs Quantum Lab

Let learners experimentally compare classical search/computation with the relevant quantum algorithm and visualize scaling or behavior.

### 7. Adaptive Learning

Use mistakes and prerequisite concepts to recommend the next experiment or mini-lesson instead of only following a fixed sequence.

### 8. Multi-Backend Comparison

Represent circuits through a common internal representation and execute them through multiple supported frameworks where feasible.

### 9. Instructor Misconception Dashboard

Show instructors which concepts and circuit patterns generate the most mistakes, attempts and hints.

# 6 Recommended MVP

The MVP should be narrow enough to finish but deep enough to demonstrate the product thesis.

- Authentication and basic learner profile.
- Three strong beginner modules: qubits/superposition, gates, and entanglement.

- A visual circuit builder.
- Code editor with a first simulator backend, preferably Qiskit Aer.
- Measurement histogram, statevector and Bloch-sphere visualization where applicable.
- Context-aware AI tutor.
- Prediction-before-run interaction.
- A small set of quizzes and coding challenges.
- Basic progress tracking.
- One polished end-to-end demonstration.

## 7 WOW / Shortlisting Features

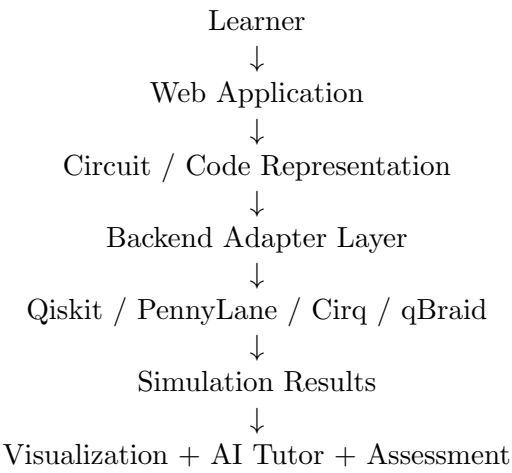
After the MVP is stable, prioritize features that make the demo memorable:

1. Circuit-aware debugging tutor.
2. What Changed? before/after visualization.
3. Prediction vs actual result comparison.
4. Interactive Noise Lab.
5. Classical vs quantum experiment.
6. Adaptive learning based on concept-level weaknesses.
7. Multi-backend execution/comparison.
8. AI-generated personalized circuit challenges.

## 8 What We Should NOT Overbuild

- A huge course library before the core interaction is polished.
- A generic ChatGPT-style chatbot disconnected from the learner's circuit.
- Implementing many quantum algorithms shallowly.
- Building a quantum simulator from scratch instead of using established frameworks.
- Overly elaborate authentication or dashboards.
- A purely cosmetic UI without measurable educational value.

## 9 Proposed Technical Architecture




The important architectural idea is to avoid tying the entire product to one framework. A common circuit representation can allow the learning layer to remain independent from individual simulation backends.

## 10 Questions to Validate With a Quantum Faculty Member

- Are our explanations of qubits, superposition, measurement and entanglement scientifically correct?
- Which quantum algorithms are appropriate for a beginner-oriented interactive platform?
- Which visualizations genuinely help students understand quantum states rather than merely looking impressive?
- Is prediction-before-execution a meaningful way to teach quantum intuition?
- How should we correctly explain amplitude, probability and measurement?
- What common misconceptions should the AI tutor detect?
- How should we introduce noise and the difference between ideal simulation and real quantum hardware?
- Which parts of the proposed platform should be treated as educationally essential?

## 11 Proposed SIH Pitch

> **We turn quantum computing from something students read about into something they experiment with.** Our platform combines interactive learning, circuit construction, simulation, visualization and a circuit-aware AI tutor into a single loop: **Learn** → **Predict** → **Build** → **Run** → **Observe** → **Explain** → **Debug** → **Challenge**.

The strongest product claim should therefore be about **learning through experimentation**, not about having an AI chatbot or another quantum simulator.

## 12 Research Sources / Starting Points

- IBM Quantum Composer documentation: https://quantum.cloud.ibm.com/docs/en/guides/composer
- Microsoft Quantum Katas: https://learn.microsoft.com/en-us/azure/quantum/katas-qdk-learning
- Microsoft Quantum Computing Fundamentals: https://learn.microsoft.com/en-us/training/paths/quantum-computing-fundamentals/
- PennyLane Codebook: https://www.pennylane.ai/codebook
- PennyLane Challenges: https://pennylane.ai/challenges/
- Quirk: https://algassert.com/quirk
- Classiq Quantum AI: https://www.classiq.io/quantum-ai

## Important Note

This is a team strategy/research draft. The SIH problem statement establishes the required capabilities; the differentiation ideas are our proposed product direction and should be validated through further competitor research and, importantly, with a quantum-domain faculty member before implementation.